Tutoriumsblatt 2

Python für AI: NumPy, APIs und der erste LLM-Call

Bearbeitungszeit etwa 90 Minuten · 40 Punkte · Lösungen im Anhang ab Seite 5

Nach diesem Blatt kannst du

- Array-Formen und Broadcasting von Hand bestimmen und mit Masken Anteile berechnen,
- Skalarprodukt, Norm und Kosinus rechnen und die Cauchy-Schwarz-Ungleichung beweisen,
- Matrix mal Vektor als Suche lesen und den Rechenaufwand abschätzen,
- HTTP, JSON und Statuscodes deuten und die Kosten eines LLM-Calls begründet senken.

So arbeitest du mit dem Blatt

Bearbeite die Aufgaben der Reihe nach, sie bauen aufeinander auf. Erlaubt sind Taschenrechner und Formelblatt. Rechne mit drei Nachkommastellen und runde erst am Ende. Code verfolgst du von Hand, programmieren musst du nicht. Bezug zum Notebook: Aufgabe 1 gehört zu TODO 1, Aufgaben 2 und 3 zu TODO 2 bis 5 und Bonus B1, Aufgabe 4 zu TODO 6, Aufgabe 5 zu TODO 7, TODO 8 und Bonus B2. Alle Messwerte sind ausgedachte Beispielwerte.

Aufgabe 1 Masken und Broadcasting von Hand

8 Punkte · etwa 16 Min.

Bezug: Modul 2, Lerneinheit 1 (NumPy: Vektorisierung, Arrays und Masken) und Lerneinheit 2 (Broadcasting)

Die Instandhaltung hat sechs Stillstände der Drehmaschine M-03 in Minuten erfasst:

```
s = np.array([30, 75, 10, 120, 45, 60])
```

(a) Gib die Ergebnisse der folgenden Ausdrücke an und sag jeweils in einem Halbsatz, was das Ergebnis bedeutet:

`s > 45` `s[s > 45]` `np.mean(s > 45)` `np.sum(s >= 45)` `s[s > 45].mean()`

[3 P]

(b) Das Array `T` enthält die Temperatur von 8 Maschinen über 24 Stunden, es hat also die Form $(8, 24)$. Bestimme mit den drei Broadcasting-Regeln für jeden Ausdruck die Form des Ergebnisses oder begründe, warum NumPy einen Fehler meldet:

(1) `m0 = T.mean(axis=0)`	(3) `T - m0`	(5) `T - m1[:, None]`
(2) `m1 = T.mean(axis=1)`	(4) `T - m1`	(6) `np.zeros(5) - np.zeros((5, 1))`

Welcher Ausdruck zieht von jeder Maschine ihr eigenes Tagesmittel ab? [3 P]

(c) Mit `grenzen = np.array([30, 60, 100])[:, None]` prüft `maske = s > grenzen` alle drei Grenzen auf einmal. Gib die Form von `maske` an, schreibe `maske` als Tabelle aus Nullen und Einsen und bestimme `maske.sum(axis=1)` und `maske.mean(axis=1)`. [2 P]

Aufgabe 2 Kosinus-Ähnlichkeit: rechnen und beweisen

9 Punkte · etwa 20 Min.

Bezug: Modul 2, Lerneinheit 3 (Skalarprodukt, Norm und Kosinus-Ähnlichkeit)

Gegeben sind die Vektoren $a = (2,\ 1,\ 2)$, $b = (1,\ 2,\ 2)$ und $c = (-2,\ 2,\ 1)$ im $\mathbb{R}^3$.

(a) Berechne $a \cdot b$, $\|a\|$, $\|b\|$, $\cos(a, b)$ und den Winkel θ zwischen a und b in Grad. Zeige, dass a und c senkrecht aufeinander stehen. Berechne außerdem $\cos(a, -2b)$. [3 P]

(b) Die Lerneinheit hat die Skaleninvarianz nur für $\lambda > 0$ gezeigt und dabei $\|\lambda b\| = \lambda\|b\|$ benutzt. Zeige direkt aus der Definition der Norm, dass für jedes $\lambda \in \mathbb{R}$ gilt: $\|\lambda b\| = |\lambda|\,\|b\|$. Folgere daraus für $\lambda \neq 0$: $\cos(a, \lambda b) = \cos(a, b)$ für $\lambda > 0$ und $\cos(a, \lambda b) = -\cos(a, b)$ für $\lambda < 0$. [2 P]

(c) Die Lerneinheit nennt die Cauchy-Schwarz-Ungleichung $|a \cdot b| \leq \|a\|\,\|b\|$ nur. Beweise sie für $b \neq 0$ in drei Schritten.

1. Betrachte $f(t) = \|a - t\,b\|^2$ für $t \in \mathbb{R}$. Multipliziere aus und zeige $f(t) = \|a\|^2 - 2t\,(a \cdot b) + t^2\|b\|^2$. Warum gilt $f(t) \geq 0$ für jedes t?
2. Setze $t^* = \dfrac{a \cdot b}{\|b\|^2}$ ein und vereinfache $f(t^*)$.
3. Folgere die Ungleichung und daraus $-1 \leq \cos\theta \leq 1$.

Zur Kontrolle mit a und b aus (a): $t^* = 8/9$ und $f(t^*) = 17/9$. [3 P]

(d) Normiere a und b auf Länge 1 und berechne $\|\hat{a} - \hat{b}\|^2$ einmal direkt und einmal mit der Formel $2 - 2\cos\theta$. [1 P]

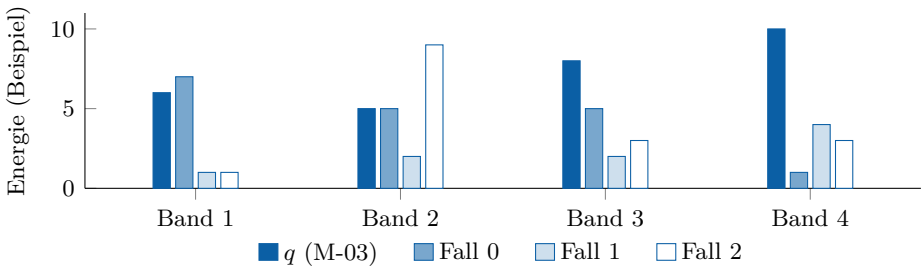
Aufgabe 3 Matrix mal Vektor: ähnliche Spektren finden

9 Punkte · etwa 20 Min.

Bezug: Modul 2, Lerneinheit 4 (Matrix mal Vektor: 10.000 Spektren durchsuchen) und Lerneinheit 2 (Broadcasting)

M-03 läuft unrund. Statt 64 Frequenzbändern nimmst du vereinfacht vier, von Band 1 (tief) bis Band 4 (hoch). Die Datenbank enthält drei historische Fälle als Zeilen von `X`, der Vektor `q` ist die aktuelle Messung an M-03.

```
X = np.array([[7, 5, 5, 1],      # Fall 0: OK
              [1, 2, 2, 4],      # Fall 1: Lagerschaden
              [1, 9, 3, 3]])     # Fall 2: Unwucht
q = np.array([6, 5, 8, 10])      # M-03, aktuelle Messung
diagnosen = np.array(["OK", "Lagerschaden", "Unwucht"])
aehnlichkeiten = (X @ q) / (np.linalg.norm(X, axis=1) * np.linalg.norm(q))
top2 = np.argsort(aehnlichkeiten)[::-1][:2]
abstand = np.linalg.norm(X - q, axis=1)
naechste = np.argsort(abstand)[:2]
```



(a) Gib die Formen von `X @ q`, `np.linalg.norm(X, axis=1)` und `X - q` an. Berechne dann `X @ q`, die drei Zeilennormen, $\|q\|$ und `aehnlichkeiten`. Zur Kontrolle: $\|q\| = 15$. [3 P]

(b) Verfolge die Zeile mit `top2` von Hand: Was liefern `np.argsort(aehnlichkeiten)`, danach `[::-1]` und danach `[:2]`? Welche Diagnose hat der ähnlichste Fall? Was käme ohne `[::-1]` heraus? [2 P]

(c) Berechne die quadrierten Abstände $\|x_j - q\|^2$ sowie `abstand` und `naechste`. Erkläre mit den Normen, warum der Abstand einen anderen Fall nach vorne holt als der Kosinus. Welche Rangfolge liefert der Abstand, wenn du vorher alle Vektoren auf Länge 1 normierst? [2 P]

(d) Rechenaufwand: In der echten Datenbank ist $N = 10\,000$ und $d = 64$. (i) Wie viele Multiplikationen braucht `X @ q`, und wie viele kommen für die Zeilennormen hinzu? (ii) Es kommen 50 Suchanfragen mit verschiedenen `q`. Vergleiche den Aufwand, wenn die Zeilennormen bei jeder Anfrage neu berechnet werden, mit dem Aufwand, wenn sie einmal vorab gespeichert werden. Zähle dabei nur die Terme der Größe $N \cdot d$. [2 P]

Aufgabe 4 Web-API: Anfrage, Antwort und Statuscodes

7 Punkte · etwa 15 Min.

Bezug: Modul 2, Lerneinheit 5 (Web-APIs, HTTP und JSON)

Für den Heizbedarf der Werkshalle holt das Notebook Tagesmittel von Open-Meteo.

(a) Nach `antwort = requests.get(URL, params=params, timeout=10)` zeigt `antwort.url` die folgende Adresse (hier auf zwei Zeilen umbrochen, `%2F` ist der kodierte Schrägstrich):

```
https://api.open-meteo.com/v1/forecast?latitude=49.01&longitude=8.4
  &daily=temperature_2m_mean&past_days=4&forecast_days=1&timezone=Europe%2FBerlin
```

Warum passt hier die Methode GET und nicht POST? Schreibe das Dictionary `params` auf, aus dem `requests` diese Adresse gebaut hat. Wie viele Tageswerte erwartest du in der Antwort? [2 P]

(b) Die Antwort lautet gekürzt, mit Beispielwerten:

```
{"latitude": 49.0, "longitude": 8.4, "timezone": "Europe/Berlin",
 "daily": {"time": ["2026-10-05", "2026-10-06", "2026-10-07",
                    "2026-10-08", "2026-10-09"],
           "temperature_2m_mean": [12.4, 16.2, 9.8, 15.0, 7.6]}}
```

Nach `daten = antwort.json()`: Gib Python-Ausdrücke für die Liste der Tagesmittel und für das Datum des dritten Tages an. Nenne den Python-Typ von `daten`, `daten["daily"]` und `daten["daily"]["time"][2]`. Was passiert bei `daten["temperature_2m_mean"]`? Berechne zuletzt die vereinfachte Heizgradzahl $\mathrm{HGZ} = \sum_t \max\left(0,\ 15\,^{\circ}\mathrm{C} - T_t\right)$. [3 P]

(c) Ein Protokoll zeigt vier fehlgeschlagene Anfragen (Beispiel):

```
Nr.  Anfrage                                                  Status
1    GET  .../v1/forecast  mit "daily", ohne "timezone"         400
2    POST .../chat/completions  model="kit.mistral-small-5"     404
3    POST .../chat/completions  60. Anfrage in 10 Sekunden      429
4    POST .../chat/completions                                  503
```

Ordne jeden Fehler der Seite des Clients oder des Servers zu und nenne die passende Reaktion. Eine Hilfsfunktion soll fehlgeschlagene Anfragen nach kurzer Wartezeit automatisch wiederholen. Bei welchen Codes ist das sinnvoll, bei welchen nicht? Begründe. [2 P]

Aufgabe 5 Der LLM-Call: Request, Tokens und Kosten

7 Punkte · etwa 16 Min.

Bezug: Modul 2, Lerneinheit 6 (Der erste LLM-Call: Nachrichten, Tokens, Kosten) und Lerneinheit 5

Für den Schichtbericht soll eine Störungsmeldung zusammengefasst werden. Ein Teammitglied schreibt den Call ohne SDK direkt mit `requests`:

```
r = requests.get(BASE_URL + "/chat/completions",
                 headers={"Authorization": "Bearer sk-7f3a9c"},
                 json={"model": "kit.mistral-small-4-119b-a8b",
                       "max_tokens": 400,
                       "messages": [{"role": "user", "content": prompt},
                                    {"role": "system", "content": SYSTEM}]})
```

(a) Der Request hat vier Schwächen. Finde drei davon, begründe jede in einem Satz und gib die Korrektur an. [2 P]

(b) Korrigiert liefert `daten = r.json()` gekürzt:

```
{"choices": [{"message": {"role": "assistant", "content": "M-03: Spindel ..."},
              "finish_reason": "stop"}],
 "usage": {"prompt_tokens": 1200, "completion_tokens": 300, "total_tokens": 1500}}
```

Gib Ausdrücke für den Antworttext und für die Zahl der Output-Tokens an. Warum schreibst du hier eckige Klammern, beim SDK-Objekt aus der Lerneinheit aber einen Punkt wie in `antwort.usage.completion_tokens`? [1 P]

(c) Berechne die kommerziellen Referenzkosten dieses Calls in Cent für Claude Haiku 4.5 (1 und 5 US-Dollar pro Million Input- bzw. Output-Tokens) und Claude Sonnet 5.5 (2 und 10 US-Dollar), Stand 04.10.2026. Welcher Anteil der Kosten entfällt auf den Output? Rechne beide Modelle auf 600 Meldungen pro Monat hoch. [2 P]

(d) Zeige allgemein: Der Output verursacht genau dann mindestens die Hälfte der Kosten, wenn $n_{\text{out}}/n_{\text{in}} \geq p_{\text{in}}/p_{\text{out}}$ gilt. Ab wie vielen Output-Tokens ist das bei 1200 Input-Tokens und den Referenzpreisen der Fall? [1 P]

(e) Zwei Sparideen für Haiku: (i) Die System-Nachricht verlangt höchstens zwei Sätze, die Antwort sinkt auf 100 Output-Tokens. (ii) Unnötige Felder fallen aus dem Prompt, der Input sinkt um 300 Tokens. Welche Idee spart mehr, und um wie viel Prozent? [1 P]

Aufgabe 6 (⋆) Vertiefung: Korrelation ist ein Kosinus

Zusatz · etwa 10 Min.

Bezug: Vertiefung zu Modul 2, Lerneinheiten 2 und 3, mit Statistik aus dem Studium

An fünf Tagen hat M-05 jeweils x_i Hundert Teile gefertigt und y_i kWh verbraucht (Beispielwerte): $x = (1,\ 2,\ 3,\ 4,\ 5)$ und $y = (3,\ 5,\ 4,\ 7,\ 6)$.

(a) Zeige: Der Korrelationskoeffizient nach Pearson ist die Kosinus-Ähnlichkeit der zentrierten Vektoren $x_c = x - \bar{x}\,\mathbf{1}$ und $y_c = y - \bar{y}\,\mathbf{1}$, wobei $\mathbf{1} = (1, \ldots, 1)$.

(b) Berechne r und vergleiche mit $\cos(x, y)$ ohne Zentrieren. Warum ist der Kosinus ohne Zentrieren so hoch?

(c) `D = np.column_stack([x, y])` hat die Form $(5, 2)$. Welche Formen haben `D.mean(axis=0)` und `D - D.mean(axis=0)`? Welche Invarianz hat r zusätzlich zur Skaleninvarianz des Kosinus?

Anhang: Lösungen

Erst selbst rechnen, dann vergleichen. Die Lösungen zeigen den Rechenweg und typische Fehler.

Lösung zu Aufgabe 1 Masken und Broadcasting von Hand

(a)

`s > 45`	`[False, True, False, True, False, True]`: welche Stillstände länger als 45 min sind
`s[s > 45]`	`[75, 120, 60]`: die langen Stillstände selbst
`np.mean(s > 45)`	$3/6 = 0{,}5$: Anteil der langen Stillstände, denn `True` zählt als 1
`np.sum(s >= 45)`	4: Anzahl mit mindestens 45 min, jetzt zählt die 45 mit
`s[s > 45].mean()`	$(75 + 120 + 60)/3 = 85$: mittlere Dauer der langen Stillstände

(b) Formen rechtsbündig untereinander, fehlende Dimensionen links als 1, je Position gleich oder eine davon 1.

(1) `m0`: Die angegebene Achse 0 verschwindet, Form $(24,)$. Das ist das Mittel jeder Stunde über alle Maschinen.

(2) `m1`: Achse 1 verschwindet, Form $(8,)$. Das ist das Tagesmittel jeder Maschine.

(3) $(8, 24)$ gegen $(24,) \to (1, 24)$: rechts $24 = 24$, links wird die 1 auf 8 gedehnt. Ergebnis $(8, 24)$.

(4) $(8, 24)$ gegen $(8,) \to (1, 8)$: rechts stehen 24 und 8, verschieden und keine ist 1. **Fehler** (operands could not be broadcast together).

(5) `m1[:, None]` hat die Form $(8, 1)$. Rechts wird die 1 auf 24 gedehnt, links $8 = 8$. Ergebnis $(8, 24)$.

(6) $(5,) \to (1, 5)$ gegen $(5, 1)$: beide Einsen werden gedehnt, Ergebnis $(5, 5)$. Kein Fehler, aber 25 Differenzen „jeder mit jedem“ statt 5 elementweise. Kein Fehler heißt nicht richtig.

Das eigene Tagesmittel jeder Maschine zieht **Ausdruck (5)** ab: Jede Zeile von `T` verliert ihren eigenen Mittelwert. Ausdruck (3) zieht dagegen das Stundenmittel ab.

(c) `grenzen` hat die Form $(3, 1)$, `s` die Form $(6,) \to (1, 6)$. Beide Einsen werden gedehnt, `maske` hat die Form $\mathbf{(3, 6)}$, eine Zeile pro Grenze:

Grenze	30	75	10	120	45	60	`sum(axis=1)`	`mean(axis=1)`
$s > 30$	0	1	0	1	1	1	4	$4/6 \approx 0{,}667$
$s > 60$	0	1	0	1	0	0	2	$2/6 \approx 0{,}333$
$s > 100$	0	0	0	1	0	0	1	$1/6 \approx 0{,}167$

Ergebnis: `maske.sum(axis=1)` $= [4, 2, 1]$, `maske.mean(axis=1)` $\approx [0{,}667;\ 0{,}333;\ 0{,}167]$. Der Wert 60 zählt bei „> 60“ nicht mit.

Typischer Fehler

Bei `s > 45` die 45 mitzählen oder bei `s > 60` die 60. Der Vergleich ist streng. Häufig ist auch die Verwechslung von `np.mean(s > 45)` (Anteil 0,5) mit `s[s > 45].mean()` (Dauer 85 min). In (b) wird Ausdruck (4) oft für richtig gehalten, weil 8 in beiden Formen vorkommt. Verglichen wird aber von rechts.

Bewertung: (a) je Ausdruck mit Deutung 0,5 P, saubere Unterscheidung Anteil, Anzahl und Dauer 0,5 P. (b) (1) und (2) zusammen 0,5 P, (3) 0,5 P, (4) mit Begründung 0,5 P, (5) 0,5 P, (6) 0,5 P, Zuordnung zu (5) 0,5 P. (c) Form 0,5 P, Tabelle 0,5 P, Summen 0,5 P, Anteile 0,5 P.

Lösung zu Aufgabe 2 Kosinus-Ähnlichkeit: rechnen und beweisen

(a) $a \cdot b = 2 \cdot 1 + 1 \cdot 2 + 2 \cdot 2 = 8, \quad \|a\| = \sqrt{4+1+4} = 3, \quad \|b\| = \sqrt{1+4+4} = 3.$

$$\cos(a,b) = \frac{8}{3 \cdot 3} = \frac{8}{9} \approx 0{,}889, \qquad \theta = \arccos(0{,}889) \approx \mathbf{27{,}3°}.$$

$a \cdot c = 2 \cdot (-2) + 1 \cdot 2 + 2 \cdot 1 = 0$, also $\cos(a,c) = 0$ und $\theta = 90°$: a und c stehen senkrecht. Mit $-2b = (-2,-4,-4)$: $a \cdot (-2b) = -16$, $\|-2b\| = 6$, also $\cos(a,-2b) = \dfrac{-16}{3 \cdot 6} = -\dfrac{8}{9} \approx -0{,}889$, der Winkel ist $180° - 27{,}3° = 152{,}7°$.

(b) Aus der Definition:

$$\|\lambda b\| = \sqrt{\textstyle\sum_i (\lambda b_i)^2} = \sqrt{\lambda^2 \textstyle\sum_i b_i^2} = \sqrt{\lambda^2}\,\sqrt{\textstyle\sum_i b_i^2} = |\lambda|\,\|b\|,$$

denn $\sqrt{\lambda^2} = |\lambda|$, nicht λ. Damit gilt für $\lambda \neq 0$:

$$\cos(a,\lambda b) = \frac{a \cdot (\lambda b)}{\|a\|\,\|\lambda b\|} = \frac{\lambda\,(a \cdot b)}{|\lambda|\,\|a\|\,\|b\|} = \frac{\lambda}{|\lambda|}\,\cos(a,b).$$

Der Faktor $\lambda/|\lambda|$ ist $+1$ für $\lambda > 0$ und -1 für $\lambda < 0$. Strecken ändert die Richtung nicht, Umklappen kehrt sie um. Das zeigt (a): $\cos(a,-2b) = -\cos(a,b)$.

(c) **Schritt 1.** Mit $\|v\|^2 = v \cdot v$ und der Symmetrie $a \cdot b = b \cdot a$:

$$f(t) = (a - tb) \cdot (a - tb) = a \cdot a - t\,a \cdot b - t\,b \cdot a + t^2\,b \cdot b = \|a\|^2 - 2t\,(a \cdot b) + t^2\|b\|^2.$$

$f(t) \geq 0$, weil $f(t)$ eine quadrierte Länge ist, also eine Summe von Quadraten.

Schritt 2. t^* ist der Scheitel der nach oben offenen Parabel, denn $f'(t) = -2\,a \cdot b + 2t\,\|b\|^2 = 0$ liefert genau t^*. Einsetzen:

$$f(t^*) = \|a\|^2 - 2\,\frac{(a \cdot b)^2}{\|b\|^2} + \frac{(a \cdot b)^2}{\|b\|^4}\,\|b\|^2 = \|a\|^2 - \frac{(a \cdot b)^2}{\|b\|^2}.$$

Schritt 3. Aus $f(t^*) \geq 0$ folgt nach Multiplikation mit $\|b\|^2 > 0$:

$$(a \cdot b)^2 \leq \|a\|^2\,\|b\|^2 \quad \Rightarrow \quad \boldsymbol{|a \cdot b| \leq \|a\|\,\|b\|},$$

weil beide Seiten nicht negativ sind und die Wurzel monoton ist. Für $a, b \neq 0$ teilst du durch $\|a\|\,\|b\| > 0$ und erhältst $|\cos\theta| \leq 1$, also $-1 \leq \cos\theta \leq 1$. Gleichheit gilt genau dann, wenn $f(t^*) = 0$, also $a = t^* b$: Die Vektoren sind parallel, und $\cos\theta = \pm 1$.

Kontrolle: $t^* = 8/9$, $f(t^*) = 9 - 64/9 = 17/9 \approx 1{,}889 \geq 0$, und tatsächlich $64 \leq 81$.

(d) $\hat{a} = \frac{1}{3}(2,1,2)$, $\hat{b} = \frac{1}{3}(1,2,2)$, also $\hat{a} - \hat{b} = \frac{1}{3}(1,-1,0)$ und

$$\|\hat{a} - \hat{b}\|^2 = \frac{1+1+0}{9} = \frac{2}{9} \approx 0{,}222, \qquad 2 - 2 \cdot \frac{8}{9} = \frac{2}{9}\ \checkmark$$

Typischer Fehler

In (b) $\|\lambda b\| = \lambda\|b\|$ auch für negatives λ schreiben. Dann wäre die Norm negativ, und der Kosinus würde sich scheinbar nicht ändern. In (c) wird $f(t) \geq 0$ oft mit „weil t beliebig ist“ begründet. Der Grund ist, dass $f(t)$ eine quadrierte Länge ist. Achte beim Winkel auf den Modus des Taschenrechners: $\arccos(8/9) \approx 0{,}476$ ist das Bogenmaß.

Bewertung: (a) Skalarprodukt und Normen 1 P, Kosinus und Winkel 1 P, Senkrecht und $\cos(a, -2b)$ 1 P. (b) Normregel mit $\sqrt{\lambda^2} = |\lambda|$ 1 P, Folgerung für beide Vorzeichen 1 P. (c) Schritt 1 mit Begründung von $f \geq 0$ 1 P, Schritt 2 1 P, Schritt 3 mit Wertebereich 1 P. (d) beide Wege je 0,5 P.

Lösung zu Aufgabe 3 Matrix mal Vektor: ähnliche Spektren finden

(a) Formen: `X @ q` ist $(3,4)$ `@` $(4,)$ $\to (3,)$, die innere 4 verschwindet. `np.linalg.norm(X, axis=1)` hat die Form $(3,)$, eine Norm pro Zeile. `X - q` ist Broadcasting $(3,4)$ gegen $(4,) \to (1,4)$, Ergebnis $(3,4)$.

$$Xq = \begin{pmatrix} 7\cdot 6 + 5\cdot 5 + 5\cdot 8 + 1\cdot 10 \\ 1\cdot 6 + 2\cdot 5 + 2\cdot 8 + 4\cdot 10 \\ 1\cdot 6 + 9\cdot 5 + 3\cdot 8 + 3\cdot 10 \end{pmatrix} = \begin{pmatrix} 117 \\ 72 \\ 105 \end{pmatrix}$$

Zeilennormen: $\sqrt{49+25+25+1} = 10$, $\sqrt{1+4+4+16} = 5$, $\sqrt{1+81+9+9} = 10$. Außerdem $\|q\| = \sqrt{36+25+64+100} = \sqrt{225} = 15$. Nenner $(150,\ 75,\ 150)$:

$$\texttt{aehnlichkeiten} = \left(\tfrac{117}{150},\ \tfrac{72}{75},\ \tfrac{105}{150}\right) = \mathbf{(0{,}78;\ 0{,}96;\ 0{,}70)}.$$

(b) `np.argsort` sortiert aufsteigend: Der kleinste Wert 0,70 steht an Index 2, dann 0,78 an Index 0, dann 0,96 an Index 1. Also `[2, 0, 1]`, umgedreht `[1, 0, 2]`, die ersten zwei `top2 = [1, 0]`. `diagnosen[top2]` ist `["Lagerschaden", "OK"]`, der ähnlichste Fall ist also ein **Lagerschaden**. Ohne `[::-1]` käme `[2, 0]` heraus, also Unwucht und OK: die beiden *unähnlichsten* Fälle.

(c) Die Zeilen von $X - q$ sind $(1, 0, -3, -9)$, $(-5, -3, -6, -6)$ und $(-5, 4, -5, -7)$. Quadrate summiert:

$$\|x_j - q\|^2 = (91,\ 106,\ 115), \qquad \texttt{abstand} \approx (9{,}54;\ 10{,}30;\ 10{,}72), \qquad \texttt{naechste} = \texttt{[0, 1]}.$$

Der Abstand holt den gesunden Fall 0 nach vorne. M-03 schwingt mit $\|q\| = 15$ dreimal so stark wie der leise Lagerschaden mit Norm 5. Der Abstand misst Unterschiede in jeder Komponente, also auch die Lautstärke, und der laute Fall 0 (Norm 10) liegt absolut näher, obwohl seine Form nicht passt. In Zahlen mit $\|x_j - q\|^2 = \|x_j\|^2 + \|q\|^2 - 2\,\|x_j\|\,\|q\|\cos\theta_j$ aus Lerneinheit 3: Fall 1 ergibt $25 + 225 - 2\cdot 5\cdot 15\cdot 0{,}96 = 106$, Fall 0 dagegen $100 + 225 - 2\cdot 10\cdot 15\cdot 0{,}78 = 91$. Der abgezogene Term wächst mit der Norm von x_j: Bei Fall 0 sind es 234, bei Fall 1 trotz des besseren Kosinus nur 144. Normiert gilt $\|\hat{q} - \hat{x}_j\|^2 = 2 - 2\cos\theta_j = (0{,}44;\ 0{,}08;\ 0{,}60)$. Die Rangfolge ist dann 1, 0, 2, genau wie beim Kosinus.

(d) (i) `X @ q` braucht $N \cdot d = 10\,000 \cdot 64 = \mathbf{640\,000}$ Multiplikationen (dazu $N(d-1) = 630\,000$ Additionen). Die Zeilennormen brauchen noch einmal $N \cdot d = 640\,000$ Quadrierungen und N Wurzeln.

(ii) Ohne Speichern: $50 \cdot 2Nd = 64\,000\,000$. Mit Speichern: einmal Nd für die Normen, dann $50 \cdot Nd$, zusammen $51 \cdot Nd = 32\,640\,000$. Das Verhältnis ist $32{,}64/64 \approx 0{,}51$, der Aufwand **halbiert sich fast**. Mit den kleinen Termen (N Multiplikationen im Nenner, d für $\|q\|$ je Anfrage) sind es 64 503 200 gegen 33 143 200, also 0,514. Ab Modul 4 und 7 normiert man deshalb die gespeicherten Vektoren einmal vorab.

Typischer Fehler

`X / np.linalg.norm(X, axis=1)` statt erst `X @ q` zu bilden: $(3, 4)$ gegen $(3,)$ vergleicht rechts 4 mit 3 und scheitert. Ebenso scheitert `q @ X`, weil die inneren Dimensionen 4 und 3 nicht passen. Beim Sortieren wird oft vergessen, dass `argsort` Indizes liefert und aufsteigend sortiert.

Bewertung: (a) drei Formen 1 P, Xq 1 P, Normen und Ähnlichkeiten 1 P. (b) Trace der drei Schritte 1 P, Diagnose und Ergebnis ohne Umdrehen 1 P. (c) Abstände und `naechste` 1 P, Erklärung mit Normen und Rangfolge nach Normierung 1 P. (d) (i) 1 P, (ii) 1 P. Folgefehler aus (a) werden nicht doppelt abgezogen.

Lösung zu Aufgabe 4 Web-API: Anfrage, Antwort und Statuscodes

(a) **GET** passt, weil nur Daten abgerufen werden und die wenigen Parameter in die URL passen. `POST` braucht man, wenn Daten im Body gesendet werden, etwa die Nachrichten beim LLM-Call in Aufgabe 5. Das Dictionary war

```
params = {"latitude": 49.01, "longitude": 8.4, "daily": "temperature_2m_mean",
          "past_days": 4, "forecast_days": 1, "timezone": "Europe/Berlin"}
```

(`8.40` statt `8.4` ist dieselbe Zahl.) Erwartet sind $4 + 1 = \mathbf{5}$ Tageswerte: vier vergangene Tage und heute.

(b) `daten["daily"]["temperature_2m_mean"]` liefert `[12.4, 16.2, 9.8, 15.0, 7.6]`, und `daten["daily"]["time"][2]` liefert `"2026-10-07"` (dritter Tag, Index 2). Typen: `daten` ist ein `dict`, `daten["daily"]` ebenfalls ein `dict`, `daten["daily"]["time"][2]` ein `str`. `daten["temperature_2m_mean"]` löst einen `KeyError` aus, weil der Schlüssel eine Ebene tiefer unter `"daily"` liegt.

Heizgradzahl: $15 - T_t = 2{,}6;\ -1{,}2;\ 5{,}2;\ 0{,}0;\ 7{,}4$. Das Maximum mit 0 macht aus $-1{,}2$ eine 0:

$$\text{HGZ} = 2{,}6 + 0 + 5{,}2 + 0 + 7{,}4 = \mathbf{15{,}2} \text{ Kelvin mal Tag.}$$

(c)

1	400, Client	Parameter fehlt: Für Tageswerte verlangt Open-Meteo `"timezone"`. Anfrage korrigieren.
2	404, Client	Unbekannte Ressource, hier ein falscher Modellname. Namen korrigieren, etwa `kit.mistral-small-4-119b-a8b`.
3	429, Client	Zu viele Anfragen in kurzer Zeit. Der Server ist in Ordnung: warten, dann langsamer erneut senden.
4	503, Server	Problem beim Anbieter (5xx). Später erneut versuchen.

Automatisch wiederholen ist bei **429 und 5xx** sinnvoll, denn diese Fehler sind vorübergehend. Bei **400 und 404** liefert dieselbe Anfrage wieder denselben Fehler. Die Wiederholung verbraucht nur Anfragen und versteckt einen Fehler, den du im Code beheben musst. Das SDK wiederholt bestimmte fehlgeschlagene Anfragen übrigens schon selbst (`max_retries`).

Typischer Fehler

Bei der Heizgradzahl das Maximum vergessen: Dann zieht der warme Tag 1,2 ab, und es kommt 14,0 heraus. Wer den Betrag nimmt, erhält 16,4. In NumPy ist `np.maximum(0, 15 - temps)` richtig. `np.max(0, 15 - temps)` scheitert mit einem `TypeError`, weil `np.max` den größten Eintrag *eines* Arrays sucht.

Bewertung: (a) Begründung für GET und Dictionary 1 P, Anzahl der Tageswerte 1 P. (b) beide Ausdrücke 1 P, Typen und `KeyError` 1 P, Heizgradzahl 1 P. (c) Zuordnung und Reaktion für alle vier Zeilen 1 P, Regel zum Wiederholen mit Begründung 1 P.

Lösung zu Aufgabe 5 Der LLM-Call: Request, Tokens und Kosten

(a) Die vier Schwächen, drei davon genügen:

1. `requests.get` statt `requests.post`: Ein LLM-Call schickt Daten im Body, das ist ein `POST`-Request an `.../chat/completions`.
2. Der Schlüssel steht im Code. Er gehört in ein Colab-Secret, gelesen mit `userdata.get("KIT_TOOLBOX_KEY")`, im Header dann als `f"Bearer {API_KEY}"`. Ist er schon gespeichert oder geteilt, löschst du ihn in der Toolbox und erzeugst einen neuen.
3. Die System-Nachricht steht hinter der Nutzer-Nachricht. Sie gehört an den Anfang der Liste, denn sie legt Rolle und Regeln fest.
4. Es fehlt `timeout`. Bei einem hängenden Server wartet das Programm sonst ewig, besser `timeout=30`.

(b) Die beiden Ausdrücke lauten:

```
daten["choices"][0]["message"]["content"]     # Antworttext
daten["usage"]["completion_tokens"]           # 300 Output-Tokens
```

`r.json()` macht aus dem JSON-Objekt ein Dictionary, dessen Werte du über Schlüssel in eckigen Klammern liest. Das SDK liefert dagegen ein Objekt mit Attributen, auf die du mit Punkt zugreifst.

(c) Mit $K = (n_{\text{in}}\, p_{\text{in}} + n_{\text{out}}\, p_{\text{out}})/10^6$ in US-Dollar:

$$K_{\text{Haiku}} = \frac{1200 \cdot 1 + 300 \cdot 5}{10^6} = \frac{2700}{10^6} = 0{,}0027 \text{ USD} = \mathbf{0{,}27} \text{ Cent},$$
$$K_{\text{Sonnet}} = \frac{1200 \cdot 2 + 300 \cdot 10}{10^6} = \frac{5400}{10^6} = 0{,}0054 \text{ USD} = \mathbf{0{,}54} \text{ Cent}.$$

Output-Anteil: $1500/2700 \approx \mathbf{55{,}6}\,\%$, bei beiden Modellen gleich, weil beide Preise verdoppelt sind. Die 300 Output-Tokens sind nur ein Fünftel aller Tokens, verursachen aber mehr als die Hälfte der Kosten. Hochgerechnet auf 600 Meldungen: Haiku $600 \cdot 0{,}27$ Cent $= 162$ Cent $= \mathbf{1{,}62}$ USD, Sonnet $600 \cdot 0{,}54$ Cent $= \mathbf{3{,}24}$ USD pro Monat.

(d) Der Output-Anteil ist mindestens $\frac{1}{2}$ genau dann, wenn

$$n_{\text{out}}\, p_{\text{out}} \geq \tfrac{1}{2}(n_{\text{in}}\, p_{\text{in}} + n_{\text{out}}\, p_{\text{out}}) \iff n_{\text{out}}\, p_{\text{out}} \geq n_{\text{in}}\, p_{\text{in}} \iff \frac{n_{\text{out}}}{n_{\text{in}}} \geq \frac{p_{\text{in}}}{p_{\text{out}}}.$$

Mit $p_{\text{in}}/p_{\text{out}} = 1/5$ und $n_{\text{in}} = 1200$: ab $n_{\text{out}} = 1200/5 = \mathbf{240}$ Output-Tokens.

(e) (i) $1200 \cdot 1 + 100 \cdot 5 = 1700$, also 0,17 Cent statt 0,27 Cent: Ersparnis $1000/2700 \approx \mathbf{37}\,\%$ (im Monat 1,02 statt 1,62 USD). (ii) $900 \cdot 1 + 300 \cdot 5 = 2400$, also 0,24 Cent: Ersparnis $300/2700 \approx 11\,\%$. Idee (i) spart mehr, weil jedes Output-Token fünfmal so viel kostet. Kürzere Antworten verkürzen außerdem die Wartezeit.

Typischer Fehler

Dollar und Cent verwechseln: 0,0027 ist der Betrag in US-Dollar. Für Cent multiplizierst du mit 100, sonst liegst du um den Faktor 100 daneben. Wer die Division durch 10^6 vergisst, erhält absurde 2700 US-Dollar für einen Call.

Bewertung: (a) je gefundener Schwäche mit Begründung und Korrektur $\frac{2}{3}$ P, höchstens 2 P. Andere stichhaltige Schwächen zählen ebenfalls, etwa dass der Statuscode nicht mit `raise_for_status()` geprüft wird. (b) beide Ausdrücke mit Erklärung 1 P. (c) Kosten beider Modelle 1 P, Output-Anteil und Monat 1 P. (d) Herleitung $0{,}5$ P, Schwelle 240 $0{,}5$ P. (e) Vergleich mit beiden Prozentwerten 1 P.

Lösung zu Aufgabe 6 Vertiefung: Korrelation ist ein Kosinus

(a) Mit $x_c = x - \bar{x}\,\mathbf{1}$ und $y_c = y - \bar{y}\,\mathbf{1}$ ist der Zähler von r genau ein Skalarprodukt und der Nenner ein Produkt zweier Normen:

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\,\sqrt{\sum_i (y_i - \bar{y})^2}} = \frac{x_c \cdot y_c}{\|x_c\|\,\|y_c\|} = \cos(x_c, y_c).$$

Daraus folgt auch $-1 \le r \le 1$ nach Cauchy-Schwarz aus Aufgabe 2.

(b) $\bar{x} = 3$, $\bar{y} = 5$, $x_c = (-2, -1, 0, 1, 2)$, $y_c = (-2, 0, -1, 2, 1)$. Dann $x_c \cdot y_c = 4 + 0 + 0 + 2 + 2 = 8$, $\|x_c\|^2 = 10$, $\|y_c\|^2 = 10$, also $r = 8/10 = \mathbf{0{,}8}$. Ohne Zentrieren: $x \cdot y = 83$, $\|x\|^2 = 55$, $\|y\|^2 = 135$, also $\cos(x, y) = 83/\sqrt{7425} \approx 0{,}963$. Beide Vektoren haben nur positive Einträge und zeigen grob in Richtung $(1, 1, 1, 1, 1)$. Deshalb ist der Kosinus hoch, fast unabhängig vom Zusammenhang. Dasselbe passiert bei Spektren, in denen überall die tiefen Bänder dominieren.

(c) `D.mean(axis=0)` hat die Form $(2,)$, die Spaltenmittel $(3, 5)$. `D - D.mean(axis=0)` ist Broadcasting $(5, 2)$ gegen $(2,)$ mit Ergebnis $(5, 2)$, jede Spalte hat danach Mittelwert 0. Der Kosinus ist invariant gegen positives Strecken, r zusätzlich gegen **Verschieben**: Mit $y + 10$ bleibt $r = 0{,}8$, der Kosinus ohne Zentrieren sinkt dagegen auf etwa 0,933.